CoCoA-Flow: Careful AI Workflows for Scientific Code
A File-by-File Guide from One User Request to One Landed Commit

V. Miranda

Code documentation manuscript

(Dated: Working draft)

This manuscript follows one requested change through CoCoA-Flow, the AI ticket system stored under `ai/` in the CoCoA SONIC repository. The system lets three AI roles—an Architect that plans and decides, an Implementer that edits and tests, and an optional Red Team that reviews—cooperate on one code change while a Python program called the watcher carries their messages, launches their sessions, and performs the one guarded Git step that publishes an accepted change. The manuscript explains the file mailbox, the message grammar, the handoff contracts, the guards that protect the backlog and the permanent notes, the candidate-to-landing Git mechanism, crash recovery, and the provider integrations that let a Claude, Codex, or Ollama-served model fill a role. The emphasis is operational: every record is assigned an exact file location, an owner, and a validity rule, and every non-obvious Python, Git, or operating-system mechanism is defined where it first protects the workflow. The companion manuscript for the emulator library itself is `documentation/emulator_code_guide.pdf`; this guide teaches the machinery that changes that library safely.

CoCoA-Flow as a workshop with three stations. A planner, a builder, and an auditor work at separate stations, joined by a stream of carried messages. The image is a frontispiece and naming illustration: the three stations are the Architect, Implementer, and Red Team roles, and the message stream is the file mailbox that this manuscript dissects. The artwork is included at its original aspect ratio. LaTeX changes only its displayed size.

I. HOW TO READ THE PROGRAM

The ticket system is a sequence of custody boundaries. A requested change begins as a sentence typed by the user, becomes a numbered message file in a folder, becomes a launched AI session inside an isolated Git worktree, becomes a saved commit called the candidate, survives an audit, and is finally published as a second commit called the landing on the branch `main`. The change is trustworthy only if its identity, scope, and evidence survive every boundary: the same ticket, the same files, the

same commit, and the same decision must be recoverable from disk after any crash, at any boundary.

This manuscript follows the executable order. Each section covers the same five checks:

1. Identify the records that exist on disk before the operation.
2. Name the function or module that owns the operation.
3. Identify the new records that exist afterward.
4. Record which step is atomic, which is retried, and which is refused rather than guessed.
5. State the check that verifies custody was preserved.

The same discipline that the emulator guide applies to tensors—shape, dtype, device, owner—is applied here to workflow records: location, format, owner, and validity rule. Nothing in this system is an in-memory agreement between processes. Every agreement is a file, and every file has exactly one writer.

A. The notation used throughout

Single capital letters name the saved Git versions that the workflow manipulates. They are labels used in code, logs, and this manuscript, not Git terminology.

A few recurring words carry exact meanings:

Ticket.: One requested change described by one source note. Small enough for one clear plan and one final decision.

Watcher, daemon.: Two names for the same Python program, `ai/tools/mailbox_daemon.py`. The user documentation says watcher; the code and its part files say daemon. It carries messages, launches role sessions, enforces guards, and performs the one Git publication step. It makes no design decisions.

Role.: A stable job description—Architect, Implementer, or Red Team—with fixed authority. A model (Claude, Codex, or an Ollama-served open-weight model) is assigned to a role per run. Authority never follows the model.

Cycle.: One completed ticket, from the Architect's plan to the recorded landing and, when the Red Team is enabled, that landing's review. The `-cycle` option counts these.

Handoff.: A structured message one role writes for another, validated against a contract before it is delivered.

Guard.: A small program that checks one custody rule and refuses on failure. Guards report facts; they never make the accept-or-reject decision, which belongs to the Architect.

Seal.: A recorded SHA-256 fingerprint that freezes one file's exact bytes until the owner deliberately reseals it.

B. Authority is fixed; models are interchangeable

The system exists because one powerful model is too expensive to use for every part of development. Planning and review can be short; reading files, writing code, and running tests consume most of an account's allowance. CoCoA-Flow therefore splits the work by responsibility and lets each responsibility use a different model:

Architect.: Turns the user's request into complete, ordered instructions; audits the Implementer's exact saved commit; owns every `GO` or `NO-GO` decision; owns the backlog and the protected notes. The user talks only to the Architect.

Implementer.: Follows the ordered plan, edits only the named files in an isolated worktree, runs the named tests, and returns the candidate commit with real command output. It never edits the backlog, the permanent notes, or anything under `ai/tools/`.

Red Team.: Optional and advisory. After a landing it reviews that exact commit and returns `NO CHANGE` or `REOPEN` with evidence. It can also run a bounded discovery search when the Architect requests one. It never edits source and cannot veto a landing.

Three consequences shape all of the code that follows. First, because the user talks only to the Architect, every other message is role-to-role and must be machine-validated: no human proofreads an Architect-to-Implementer handoff, so `handoff_contract.py` does. Second, because the Implementer may be a smaller or open-weight model, the plan must remove all design freedom; the contract literally rejects a directive that leaves a choice open. Third, because roles hold different authority, the tools must know which process is which: role identity travels in environment variables and message headers, and several commands refuse to run outside the role they serve.

One more actor appears in the code but is not a role: the watcher itself. It has authority over exactly one Git action—building and publishing landing L—and that authority is hedged by checks described in Section XV. Everywhere else it moves files and launches processes.

C. Mechanics that the later code uses

The emulator guide defines Python object mechanics because tensors flow through them. Here the flowing objects are processes, files, and commits, so the load-bearing mechanics come from the operating system and Git. Each definition below is used repeatedly in later sections.

Symbol	Plain meaning	Program owner
C	Candidate: the Implementer's exact proposed commit for one ticket	Created by the Implementer; audited by the Architect; never rewritten
L	Landing: the accepted commit the watcher builds on top of current `main` after a `GO`	Created and published by the watcher alone
M0, M1	The `main` commit L was built on, and a newer `main` commit that appeared afterward	Git; the stale-integration check compares them
B, P	For a protected-note update: `main` before the edit, and the Architect's clean note commit whose only parent is B	Architect creates P; the watcher verifies the pair
D0, D1	The controller program on `main`, and a proposed replacement controller under test	`control_plane_handoff.py` rehearses D1 against copies of D0's saved state

TABLE I. Commit labels used throughout the manuscript. Each label names one exact 40-character Git commit identifier recorded in a workflow file. The labels appear verbatim in daemon messages such as `MAILBOX-NOTES-COMMIT` (P) and in log lines about candidate C and landing L, so learning them once pays off in every later section.

Process.: One running program with its own memory. The watcher and every AI session are separate processes. Two processes share nothing unless they exchange it through files, pipes, or exit codes—which is why every agreement in this system is a file.

`subprocess`.: The Python module that starts another program, optionally captures what it prints, and reports its exit code. The watcher uses it to launch role sessions and to run Git. An exit code of zero conventionally means success; the watcher treats any nonzero code as failure and never guesses otherwise.

Environment variables.: Named strings a parent process hands to a child at launch. They are invisible to other processes. The watcher uses them to tell a role session who it is and where its records live (for example `MAILBOX_ROLE` and `MAILBOX_NOTES_BASE`), so a role cannot honestly mistake its own identity, and a command run outside a role process finds the variable absent and refuses.

Atomic rename.: The operating system can move a file within one disk volume so that every observer sees either the old location or the new one, never a half-copied file. The mailbox relies on this: moving a message file *is* the state change, so a crash at any instant leaves the message wholly in one folder.

SHA-256 fingerprint.: A 64-hexadecimal-character value computed from exact bytes. Changing one character of the input changes the fingerprint beyond recognition. The guards use it to freeze files (the backlog seal), to identify archives (bundles), and to bind evidence to exact text (blocked-handoff fingerprints). A fingerprint detects accident, not malice: a hostile program with the same file permissions could rewrite both the file and the recorded value, and the documentation says so plainly.

Commit.: Git's saved snapshot of the whole project, named by a 40-hexadecimal-character identifier computed from its content and history. A commit never changes; "moving" a branch means pointing the branch name at a different commit. This immutability is why the workflow can bind an approval to candidate C: the identifier in the `GO` record can only ever mean one exact tree.

Branch.: A movable name for a commit. `main` is the public sequence of accepted versions. The AI roles work on `claude/*` and `codex/*` branches that never become `main`; only landings do.

Worktree.: An additional working folder attached to the same repository, with its own checked-out branch. Git remembers its worktrees (`git worktree list`). CoCoA-Flow gives each role its own worktree so two jobs never edit through the same folder; the layout appears in Section III.

Fast-forward and squash.: A fast-forward is Git advancing a branch name along existing history without creating a merge commit. A squash combines several commits' total effect into one new commit. The watcher squashes a ticket's accepted work into one landing commit, then fast-forwards `main` to it, so `main` reads as one commit per accepted ticket.

Push.: Sending local commits to the shared repository on GitHub. A non-force push refuses to overwrite newer remote work. The watcher attempts exactly one non-force push per landing and records a debt file when the attempt fails or cannot be verified; it never retries by rebuilding the landing.

D. How this manuscript states software limits

The system's own documentation is explicit about what its checks cannot do, and this manuscript preserves those statements rather than improving on them.

Three recur. The seal chain and the note guard protect against *accidental* edits; a deliberately hostile process running as the same operating-system user could rewrite both a file and its recorded fingerprint. The Git-movement check after an Implementer turn detects that `main` moved but cannot attribute the movement. And a passing guard proves only its named fact—bytes unchanged, count within limit, sections present—never that a change is correct. The one place correctness is judged is the Architect's audit, and that judgment is deliberately left to a reasoning agent rather than a program.

II. ONE TICKET FROM REQUEST TO LANDING

Before dissecting modules, walk one ordinary ticket through the whole machine. Every clause in this walk-through is expanded by a later section; the goal here is to fix the order of events and the custody transfer at each step.

Step 1: the user writes a source note and sends one request. The source note is a Markdown file under `ai/notes/` stating the wanted behavior, the limits, and the checks that prove success. The request itself is one command:

```
python3 ai/tools/mailbox_daemon.py \
  --send architect \
  --unit "Please coordinate the ticket
          in ai/notes/version-flag.md."
```

This writes one numbered Markdown message addressed to the Architect and exits. No AI session starts. The mailbox request is only a pointer; the cited source note carries the substance, and if the two ever disagree the source note wins.

Step 2: the watcher claims the message. A separately started watcher (`-watch`) polls the mailbox every 20 seconds. Before launching any role it moves the message file into the in-progress folder with an atomic rename; the move is the claim, so a second watcher cannot start the same request.

Step 3: the Architect plans. The watcher launches an Architect session in the Architect's own worktree. The Architect turns the request into a complete implementation directive inside the ticket note—files, functions, ordered edits, tests, commands, and required helper sessions—and validates it with `handoff_contract.py` before sending. A directive that leaves a design choice open is invalid by construction.

Step 4: the Implementer builds candidate C. The watcher delivers the directive as a new message on the Implementer route, launches an Implementer session in the isolated Implementer worktree, and records the Git state of `main` and the user's checkout before the turn. The Implementer follows the plan, runs the named tests, and commits the complete proposed change: candidate C. Its handoff back to the Architect reports the commit, the evidence, and the helper-session results the plan required.

Step 5: the Architect audits exactly C. The watcher gives the Architect a read-only view of the exact candidate commit. The audit reruns the guards—scope classification, character count when a limit is set, permanent-note fingerprints, backlog seal—and judges the evidence. `NO-GO` returns focused repair instructions to the Implementer. `GO` produces a decision record that names C by its full 40-character identifier, closes and reseals the backlog entry, and ends the Architect session.

Step 6: the watcher lands. Only after the Architect process has exited does the watcher build landing L: one squash commit with the ticket's total effect, placed directly after the current tip of local `main`, carrying the Architect-approved commit message plus two machine-readable recovery labels. It verifies that the user's `main` folder is clean and unmoved, fast-forwards `main` to L, updates every idle, clean role worktree to L, and makes one non-force push attempt (or records the user's explicit `-github no` choice). Any failed check preserves C, the `GO`, and L on disk and changes nothing in the user's folder.

Step 7: the Red Team reviews exactly L. When enabled, the Red Team reads a temporary read-only checkout of L and returns `NO CHANGE` or `REOPEN` with evidence. A `REOPEN` forces one more Architect decision—accept the evidence and reopen the ticket, or refuse it and bar that objection permanently—and that decision is committed to the backlog before the cycle is allowed to end.

The rest of the manuscript expands these steps in executable order: the on-disk world the steps read and write (Section III), the message grammar (Section VII), the store and watch loop (Section VIII), dispatch and providers (Section XI), the contracts (Section XII), the guards (Section VI), the landing (Section XV), recovery (Section XVI), and the ticket lifecycle rules (Section XVIII).

III. THE ON-DISK WORLD

Every later module is easier to read once the disk layout is fixed in mind, because each module owns a small part of it and refuses to touch the rest. This section tours the layout as it exists on a working installation.

A. The repository side: code, notes, and the backlog

`ai/tools/`: The twenty-five Python files this manuscript teaches. Only `mailbox_daemon.py` is run directly; the other `mailbox_*.py` files are parts of that one program (Section IV). The folder is globally protected: no mailbox ticket may change a file here, and three independent checks enforce that (Section XVII).

`ai/notes/`: Working records. Permanent knowledge lives in exactly eleven Markdown notes; `backlog.md` is the tracked ticket ledger; temporary ticket notes, review notes, and source notes come and go untracked.

`ai/notes/role-contract.yaml`: The machine-readable role contract, examined in Section V. Despite the name it is stored as JSON-compatible YAML; the tools read one copy at startup.

`ai/notes/backlog.md` **and its seal:** The backlog's companion records `.backlog-guard.json` (the sealed SHA-256 fingerprint) and `.backlog-guard.lock` sit beside it, ignored by Git. Both filenames appear in the role contract's forbidden list, so a candidate that touches either is refused.

B. The mailbox: folders whose names are states

The mailbox lives at `ai/notes/mailbox/`. A message's location *is* its state:

the mailbox root: Waiting messages, one numbered Markdown file each. The number prefix orders messages within a lane (Section VIII).

`inflight/`: Messages currently being handled. The watcher moves a message here—an atomic rename—immediately before launching the role, so the move doubles as the claim.

`done/`: Messages whose role turn finished and whose final move the watcher confirmed.

`failed/`: Messages whose turn failed, was refused, timed out, or was killed; they can be requeued by moving the file back.

`.dispatch-history/`: Timeout and dispatch records kept for diagnosis.

lock files: `.dispatch.lock`, `.sequence.lock`, and `.ticket-cycle-state.lock` serialize the watcher's own critical sections: claiming work, numbering messages, and updating the ticket cycle record.

A message left in `inflight/` after a crash is deliberately *not* cleaned up by guesswork: the watcher could not confirm the outcome, so the file stays for inspection, and later work needing the same role folder waits (Section XVI).

C. Relay logs

Every launched role session streams its output to a timestamped log under `ai/notes/relay/`, for example

```
ai/notes/relay/
  20260713-231623-dispatch-opus.log
```

The watcher prints only essential status lines to the terminal and names the log for anyone who wants the full stream—the same essential-only output policy the emulator library uses.

D. The worktree side: one folder per role

Git worktree state lives under `.claude/worktrees/`:

```
.claude/worktrees/
  mailbox-primary/        Architect
  mailbox-implementer/    Implementer
  mailbox-sol/            Red Team
  .mailbox-primary-worktree.json
  .mailbox-implementer-worktree.json
  .mailbox-sol-worktree.json
  .mailbox-primary-worktree.lock
  .main-checkout-turn.lock
```

Each JSON state file records the worktree's `name`, `path`, `branch`, owning `repository`, `topology`, and a `schema` number. The schema number is how an old installation is recognized: a state file written by an older tool version is refused with an error that literally names `schema-1`, and the recovery appendix of the tools guide walks the user through upgrading rather than deleting. The role contract pins the branch names: the two Claude roles ride `claude/mailbox-primary` and `claude/mailbox-implementer`; the Red Team worktree rides `codex/mailbox-sol`. The topology string `separate-role-worktrees-v1` names the whole arrangement, so a future rearrangement is an explicit migration rather than a silent drift.

The user's own checkout is not in this list. No ordinary AI job ever starts there; the watcher enters it only for the final guarded fast-forward of `main`, and `.main-checkout-turn.lock` serializes even that.

IV. ONE PROGRAM IN TWELVE FILES: THE DAEMON SPLIT

The first fact a reader needs about the code's shape: `mailbox_daemon.py` and the eleven `mailbox_*.py` files beside it are *one program*. The daemon's docstring says it directly: "A part holds one area's functions (worktrees, watch control, message envelopes, dispatch, landing, recovery, and so on); this file keeps every constant and runtime setting, loads each part at start, and stays the one namespace through which every function is reached. Run only this file."

A. How a part is adopted

A part file is not imported. It contains *no* module-level `import` statements at all—not even `import os`. Each begins with `daemon = None` and a `PART_EXPORTS` tuple naming what it contributes. At startup the daemon executes each part and adopts it:

```
part.daemon = _DAEMON_NAMESPACE
for name in part.PART_EXPORTS:
    if name in globals():
        raise RuntimeError(
            "daemon part " + filename
            + " redefines " + name)
    globals()[name] = getattr(part, name)
```

Two things happen here. Every exported function is copied into the daemon's own global namespace—so `mailbox_daemon.dispatch` resolves to a function physically written in `mailbox_dispatch.py`—and the part's `daemon` name is bound to a live view of those same globals. The collision check enforces single ownership: a name that two parts both export is a startup error, because two owners for one function would make behavior depend on load order.

B. Why every reference reads `daemon.name`

Inside a part, every collaborator is reached as `daemon.`⟨`name`⟩: sibling functions, daemon constants, and even the standard library (`daemon.os`, `daemon.re`, `daemon.subprocess`). The smallest exported function shows the style:

```
def fsync_directory(directory):
    flags = daemon.os.O_RDONLY
    if hasattr(daemon.os, "O_DIRECTORY"):
        flags |= daemon.os.O_DIRECTORY
    descriptor = daemon.os.open(
        directory, flags)
    try:
        daemon.os.fsync(descriptor)
    finally:
        daemon.os.close(descriptor)
```

This looks unusual until the purpose is stated, and the `_DaemonNamespace` docstring states it: "Part code reads each collaborator as `daemon.`⟨`name`⟩ at call time. Reads and writes go straight to this module's own variables, so when a test rebinds `mailbox_daemon.`⟨`name`⟩, every part sees the new binding immediately." The daemon's roughly one hundred functions were split from one very large file into topical parts, and the test suite patches daemon attributes by name; late binding through one namespace means the split changed *where functions are written* without changing *how they are addressed*. A plain `from mailbox_store import claim_message` would freeze each caller to the original object and silently break every such patch. This is a deliberate, load-bearing design; do not "clean it up" into ordinary imports.

The view is a small class rather than the module object itself because the tests also load isolated daemon copies that never enter Python's loaded-module registry; a view over `globals()` works for those copies too:

```
def __getattr__(self, name):
    try:
        return self._module_globals[name]
    except KeyError:
        raise AttributeError(
            "mailbox daemon namespace has "
            "no name " + repr(name))
```

C. Consequences for the reader

Three practical reading rules follow. First, to find a function, grep all twelve files; to find a constant, look in `mailbox_daemon.py`, which owns every constant and runtime setting. Second, when older documentation says "the function `prepare_exact_squash_landing` in `mailbox_daemon.py`," both halves are true at once: the function is addressed through the daemon but written in `mailbox_landing.py`. Third, any new part file must join the `trusted_tools` census in the role contract (Section V), or the daemon will refuse to start.

Six smaller helpers are loaded separately from the parts— `role_contract.py`, `reopen_transition.py`, `provider_health.py`, `candidate_admission.py`, `review_dispatch.py`, and `control_plane_handoff.py` arrive via `importlib` and are stored as daemon attributes such as `_CANDIDATE_ADMISSION`. They are ordinary importable modules (several are also standalone command-line tools), which is exactly why they are not parts: a part cannot be imported by anyone else, and these helpers must also serve tests, human commands, or, in one case, a probe process running a different daemon copy.

V. THE ROLE CONTRACT: AUTHORITY STORED AS DATA

Before any message moves, every tool must agree on who may do what. That agreement is `ai/notes/role-contract.yaml`, read by the one-purpose module `role_contract.py` whose entire docstring is one sentence: "Read the small protected contract shared by the mailbox tools." The module ends by executing `ROLE_CONTRACT = load_role_contract()` at import time, so every tool that imports it either receives a fully validated contract or fails to import at all. There is no half-configured state.

Permission	A	I	RT
may_decide	yes	no	no
may_edit_source	no	yes	no
may_edit_backlog	yes	no	no
may_edit_protected_policy	yes	no	no
may_land	no	no	no

TABLE II. The role permission matrix stored in `role-contract.yaml`. Columns: A the Architect, I the Implementer, RT the Red Team. Note the last row: *no role* may land. The contract separately records `landing.creator = "daemon"`, so publication to `main` is a program action that no reasoning agent performs.

A. What the contract records

Despite the `.yaml` suffix the file is JSON-compatible and must be in exact canonical form (sorted keys, two-space indent, one trailing newline); the loader re-serializes what it parsed and requires byte equality, refusing otherwise with "role contract must use canonical formatting." The same canonical-encoding idiom appeared in the backlog seal state file: a format with one spelling per meaning makes every edit visible.

The contract's top level has exactly ten keys: `schema_version` (which must equal 2), `roles`, `candidate`, `landing`, `backlog`, `evidence`, `limits`, `runtime`, `protected_paths`, and `worktrees`. The role table is worth reading in full because it is the constitution of the whole system:

Three other entries recur throughout the manuscript. `candidate` pins `creator: implementer`, `full_hash_required: true`, and `immutable: true`. `landing` pins `creator: daemon`, `parent_count: 1` (a landing has exactly one parent), and `force_push_allowed: false`. `protected_paths` lists the files and prefixes no candidate may touch—`ai/tools/`, `.claude/`, `.codex/`, the mailbox and relay folders, the backlog and its guard files—plus the census of the eleven permanent notes, the three role files, and `trusted_tools`: a name-to-path map of all the tool files, which is how a part file proves it belongs to the trusted set.

B. The compiled safety floor

Here is the design idea a first reading can miss. The YAML is editable (by the Architect, through protected-policy administration), so what stops an edited contract from granting new authority? The reader module carries a compiled copy of the non-negotiable entries and requires the file to match it:

```
for section, required in \
        safety_floor.items():
    if value[section] != required:
        raise RoleContractError(
            section + " does not match "
            "the compiled safety floor")
```

The floor pins the role permission matrix above, `candidate.creator`, `landing.creator`, `landing.parent_count`, `force_push_allowed = False`, and `red_team_advisory = True`, and the `trusted_tools` map must equal the bootstrap identities compiled into the reader. An edited YAML can therefore *tighten* configurable settings (add a forbidden prefix, change a timeout) but can never make the Implementer a decider or allow a force push: those changes require editing the Python reader itself, which lives in `ai/tools/`—the folder no ticket can change (Section XVII). Configuration and authority are deliberately different substances.

The loader applies the same paranoid-read idioms as the guards: refuse a symlink or non-regular file, cap the byte size before and after reading, decode UTF-8 strictly, and parse JSON with a hook that refuses duplicate keys:

```
def _object(pairs):
    result = {}
    for key, value in pairs:
        if key in result:
            raise RoleContractError(
                "duplicate key: "
                + repr(key))
        result[key] = value
    return result
```

Duplicate keys matter because a JSON parser's default behavior is last-key-wins, which would let a file present one value to a casual reader and a different value to the program.

VI. GUARDS: SMALL PROGRAMS THAT CHECK ONE CUSTODY RULE

Three standalone guards protect the records that roles must not drift: the backlog seal, the permanent-note census, and the ticket character count. Each follows the same design: it verifies one narrow fact, it prints a machine-greppable verdict line, it exits 0 on success and a distinct nonzero code on refusal, and it never makes the accept-or-reject decision. Reading one guard closely teaches the idioms the others reuse.

A. The backlog seal: `backlog_guard.py`

The problem: only the Architect may edit `ai/notes/backlog.md`, but the Implementer and Red Team read it and could—by defect or hallucination—write to it. The guard's own docstring states the threat model exactly:

> This is an accidental-change guard, not an authorization system. A malicious program that can rewrite both local files can defeat it. Its purpose is to make an Implementer or Red Team hallucination visible before the Architect continues working.

The mechanism is one ignored state file, `ai/notes/.backlog-guard.json`, holding the SHA-256 the Architect last accepted. Three subcommands manage it:

`check`: Read-only. Recomputes the backlog's SHA-256 and compares it with the saved value. The refusal is exact and symmetric:

```
observed = _sha256(backlog)
expected = state["sha256"]
if observed != expected:
    raise GuardError(
        "backlog SHA-256 mismatch; "
        "accepted=" + expected
        + " observed=" + observed)
return expected
```

`initialize -architect-ack`: Creates the first state file for an existing backlog.

`seal -previous-sha256 SHA256 -architect-ack`: Accepts one deliberate Architect edit. The caller must present the value that `check` printed *before* the edit; a stale or guessed value is refused with instructions:

```
"--previous-sha256 does not match the
 saved state; run check before editing
 and copy its accepted SHA-256"
```

The `seal` requirement builds a chain: each accepted state records the fingerprint it replaced (`previous_sha256`), so sealing forces the Architect through check-then-edit-then-seal in that order. One deliberate wrinkle rewards close reading: if a previous `seal` wrote the new state and crashed before reporting success, the retry presents a `-previous-sha256` that no longer matches. The guard tolerates exactly that case—saved state already sealed, its recorded predecessor equal to the presented value, and the backlog still verifying—and treats the retry as a harmless no-op. Crash-retry idempotence is a theme this manuscript will meet again in the landing and recovery sections.

Two defensive idioms in this small file recur across the toolkit. First, canonical encoding: the state file is written as `json.dumps(document, indent=2, sort_keys=True)` plus one newline, and the parser requires the stored bytes to equal that canonical form exactly—so even an edit that preserves JSON meaning (reordered keys, changed whitespace) is visible. Second, paranoid reads: the file reader refuses symlinks, files with multiple hard links, oversize files, and any file whose size or modification stamp changes while it is being read. Both idioms defend against accident and mid-read races, not against a hostile peer, and the docstring says so.

Authority is checked before writing. When the environment variable `MAILBOX_ROLE` identifies a role, only `architect` may seal; outside any role process, the explicit `-architect-ack` flag is the human's acknowledgment. The refusal reads:

```
"MAILBOX_ROLE identifies " + role
+ "; only the Architect may replace
   the saved backlog SHA-256"
```

One relationship must be stated precisely because a careless reading gets it wrong: the watcher does *not* shell out to `backlog_guard.py`. The daemon has its own internal sealed-backlog verifier (`_validate_sealed_backlog`, defined in `mailbox_store.py`) that reads the same two files and enforces the same convention. The standalone guard is the Architect's and the user's hand tool; the daemon reimplements the verification at every point where it is about to trust backlog contents. What connects them is the shared state-file convention, not a call edge.

B. The permanent-note census: `permanent_note_guard.py`

Eleven Markdown notes, the role contract, and a small protected reference catalog are the repository's durable policy. The Implementer and Red Team never edit them. Before handing work to another role and again before the final `GO`, the Architect runs this guard with the ticket's pinned starting commit:

```
BASE="$(git rev-parse HEAD)"
python3 ai/tools/permanent_note_guard.py \
  --repo "$PWD" \
  --base "$BASE"
```

The design premise appears in the docstring: "The expected SHA-256 values are calculated from Git. They are not stored in an editable checksum file." There is nothing to tamper with except Git history itself. For every protected file the guard reads four states and requires all four to match the pinned base:

```
for path in PROTECTED_POLICY_FILES:
    expected = _git_bytes(repo, base, path)
    head = _git_bytes(repo, "HEAD", path)
    staged = _git_bytes(repo, "", path)
    working = _working_bytes(repo, path)
    _require_same(path, expected, head,
                  "current HEAD")
    _require_same(path, expected, staged,
                  "Git staging area")
    _require_same(path, expected, working,
                  "working tree")
    rows.append((_sha256(expected), path))
```

Comparing four states is the point: an accidental edit cannot hide by being staged or committed, because the staging area and `HEAD` are checked alongside the working file. The staged copy is read with a small Git idiom worth knowing: `git show :path` with an empty object name reads from the index rather than a commit.

Three further defenses reward attention. First, the census is exact: base, `HEAD`, and index must each contain *exactly* the eleven notes—a missing note and an extra note are both refusals, listed by name—so a twelfth note cannot slip in silently, and lookalike filenames differing only by case or Unicode normalization are rejected as collisions. Second, the guard checks itself first: before trusting its own path lists it verifies that `permanent_note_guard.py` and `role_contract.py`—the two files that define what "protected" means—are themselves unchanged across `HEAD`, index, and working tree. This stops the obvious two-file attack of editing a note and the guard that would detect the edit. Third, the whole snapshot is taken twice and the two snapshots must be identical; a repository that changes mid-verification is refused with "protected state changed while it was verified" rather than reported half-old, half-new.

Success prints one `digest  path` row per file and the verdict line `PERMANENT-NOTE-GUARD PASS base=... notes=11 contract=1`; any difference exits 2. A pass proves that the protected files match the base. It approves nothing.

C. The character-count guard: `ticket_change_guard.py`

When a watch runs with `-max N`, every ticket must fit within `N` changed characters, counted as characters added plus characters removed between the ticket's base commit and its proposed result. The interesting question is what "counted exactly" means, and this module answers it with more care than a first reading expects.

What is measured. Not lines, not diff hunks: Unicode code points. For each changed file the guard computes the exact minimum number of single-character insertions and deletions that transform the old text into the new one. The tool first trims the equal prefix and suffix:

```
prefix = 0
shared_length = min(len(old_text),
                    len(new_text))
while (prefix < shared_length
       and old_text[prefix]
           == new_text[prefix]):
    prefix += 1
```

and then runs a longest-common-subsequence computation on the remaining middle. With the common subsequence length in hand the counts follow by subtraction:

```
def count_prepared_delta(prepared):
    old_length = (prepared.old_end
                  - prepared.old_start)
    new_length = (prepared.new_end
                  - prepared.new_start)
    common = exact_lcs_length(
        prepared=prepared)
    return CharacterCount(
        added=new_length - common,
        deleted=old_length - common)
```

Replacing one character therefore counts as two—one deleted, one added—exactly as the user documentation promises. When the dynamic-programming table would exceed a fixed work budget, the guard switches to an equivalent lower-memory algorithm rather than approximating; when even that budget is exceeded, it refuses with exit code 2 instead of guessing. The fixed ceilings (4 MiB per blob, 16 MiB total, bounded comparison work, 30 seconds per Git command) cannot be raised from the command line: `-max` bounds the ticket, not the guard's own appetite.

Which two versions are compared. This is where the guard encodes workflow policy. The Implementer's self-check form requires a clean worktree whose `HEAD` *is* the candidate—any staged, unstaged, or nonignored untracked change is refused, and so is any tracked file flagged with Git's `skip-worktree` or `assume-unchanged` bits, "which can hide edits." The Architect's audit form must name the exact candidate:

```
python3 ai/tools/ticket_change_guard.py \
  --repo "$PWD" \
  --base FULL_BASE_COMMIT \
  --architect-audit \
  --candidate FULL_CANDIDATE_COMMIT \
  --max 1200
```

Both commits must be spelled with all 40 hexadecimal characters; abbreviations and `HEAD` are refused in the audit form. The reason is concurrency: the Implementer may already be building ticket B while the Architect audits ticket A, so "the current `HEAD`" is a moving target, while a full commit identifier names frozen content forever. The guard also requires the base to be an ancestor of the candidate, re-checks the clean-candidate invariant *after* measuring (a mid-measurement edit would otherwise win), and refuses binary files, invalid UTF-8, and changed submodules—all things that cannot be counted as text honestly.

Which copy of the guard runs. A subtle self-defense: when the dispatch environment names the authoritative guard through `MAILBOX_TICKET_CHANGE_GUARD`, the module compares that absolute path with its own file location and refuses to run as an accidental worktree copy of itself. Role worktrees contain the whole repository, including `ai/tools/`; without this check a stale copy in an old worktree could measure with outdated rules.

Three exit codes partition the outcomes: 0 within limit (or no limit), 1 measurably over, 2 cannot measure safely. The distinction between 1 and 2 matters to the Architect: an over-limit ticket is a policy decision, an unmeasurable ticket is an evidence failure, and the guard refuses to blur them.

D. Scope classification: `candidate_admission.py`

The smallest file in the toolkit—41 lines—answers one question about a finished candidate: may it proceed normally, does it need an Architect scope decision, or did it touch a globally protected path? It is deliberately pure logic: no Git, no file reads, no state. The daemon supplies the changed paths (already read from Git) and the protected sets (already read from the role contract), and the entire decision is:

```
protected = set(protected_paths)
if protected:
    return ("PROTECTED_PATH_VIOLATION",
            protected)
exceeded = (set(changed_paths)
            - set(path_scope or ()))
if exceeded:
    return "SCOPE_EXCEEDED", exceeded
return "IN_SCOPE", set()
```

The precedence encodes policy. A protected path always wins: no ticket label, including the retired `protected-control-plane` class, can authorize a change under `ai/tools/` or the other forbidden prefixes. `SCOPE_EXCEEDED`, by contrast, is deliberately *not* a hard refusal: a candidate that edits a file the plan did not name is preserved and reported, and the Architect decides whether the expansion was a legitimate implementation discovery or a reason to send the ticket back. The difference between "refuse automatically" and "escalate to the decider" is drawn in four lines of set arithmetic, which is why the module can afford to be this small: everything it needs to know arrives as arguments, and everything it decides leaves as a label.

Why make it a separate file at all? Because the classification is policy that tests want to probe exhaustively, and a pure function with no I/O can be tested against every path combination without building a repository. The daemon wraps it and feeds it real Git output; the tests feed it adversarial lists.

VII. THE MESSAGE PROTOCOL: ENVELOPES

Everything the roles say to each other travels as Markdown files whose first lines form a machine-read header block: the *envelope*. The module `mailbox_envelopes.py` owns this grammar—building each role's preamble, parsing each envelope kind, and matching every returned block to the exact ticket it claims to answer. Its docstring states the design rule that explains most of its 1,771 lines: "refusing a mismatch instead of guessing."

A. Filenames route; headers bind

A message's recipient is encoded in its filename. The daemon recognizes waiting messages with one pattern:

```
PENDING_MESSAGE_RE = re.compile(
    r"\d+-to-(fable|opus|sol|daemon)\.md$")
```

`to-fable` is the Architect, `to-opus` the Implementer, `to-sol` the Red Team, and `to-daemon` the watcher itself. The names are stable route addresses, not model choices: an Ollama-served Implementer still reads `to-opus` files. Files ending `-to-user.md` are replies for a person and are never dispatched. The leading number, zero-padded to four digits, orders messages within a lane.

The headers then bind the message to one exact ticket. A ticket-flow exchange between Architect and Implementer must begin:

```
MAILBOX-FLOW: ticket
MAILBOX-CYCLE: TICKET-ANCHOR@FULL-COMMIT
MAILBOX-MODE: normal
```

followed by one blank line and the body. The cycle identifier joins the ticket's stable backlog anchor to its full 40-character starting commit, so every message in the exchange names both the work and the exact code version it started from. `MAILBOX-MODE` is `normal` or `two-role` and cannot change during a ticket. The parser refuses a missing or malformed header triple with "a ticket exchange needs exact MAILBOX-FLOW, MAILBOX-CYCLE, and MAILBOX-MODE headers"—and then, having matched the header, scans the *body* for a second spelling of any reserved header and refuses that too ("duplicate ticket-cycle flow header"). A body that smuggles its own `MAILBOX-CYCLE` line could otherwise change meaning depending on which line a later reader noticed first.

Each message kind has its own required envelope. The most consequential is the Architect's `GO`:

```
MAILBOX-RETURN: architect-go
MAILBOX-CYCLE: TICKET-ANCHOR@FULL-COMMIT
MAILBOX-CANDIDATE: FULL-CANDIDATE-COMMIT
MAILBOX-MODE: normal
MAILBOX-DECISION: GO
```

Note what is absent: a body. The parser refuses free-form prose in a `GO` request ("an Architect GO request may not carry free-form work"). The decision names the cycle and candidate C, and nothing else; explanations belong in notes, where they cannot be mistaken for machine instructions. The permanent-note administration return (Section XVII B) has the same shape with

`MAILBOX-BASE` (B) and `MAILBOX-NOTES-COMMIT` (P), and the parser additionally requires B ≠ P. Red Team returns carry `MAILBOX-RESULT:` with exactly `NO CHANGE` or `REOPEN`.

B. Freshness: the inode snapshot

When the watcher launches a role turn, how does it later tell which mailbox files that turn produced, as opposed to files that already existed? Not by timestamps, which clocks can skew, and not by filename guessing. Before the turn it records the inode—the filesystem's unique identity number—of every file on the relevant route; after the turn it globs again and keeps only files whose inode was not in the snapshot:

```
fresh = []
for path in glob(
        os.path.join(MAILBOX, "**",
                     pattern),
        recursive=True):
    inode = regular_inode(path=path)
    if inode is None \
            or inode in before_inodes:
        continue
    fresh.append(path)
return fresh
```

The return matchers then demand *exactly one* fresh file that names this turn's identity. Zero matches is a refusal; two matches is a refusal ("expected exactly one new same-cycle IMPLEMENTER_HANDOFF; found ..."). A `GO` whose headers name a different cycle, candidate, or mode is refused with "GO does not name this turn's exact cycle, candidate, and mode." The pattern repeats for every return kind, and it is the mechanism behind a claim made earlier: approval for one candidate can never be applied to another, because the matcher will not accept a `GO` that names anything but the exact awaited identity.

C. Semantic checks against the live repository

Envelope validation is not only textual. An Implementer handoff's declared candidate must equal the Implementer worktree's actual `HEAD` ("Candidate commit does not name the exact Opus HEAD"). A context handoff—the record a nearly-out-of-context Implementer writes before being replaced—must agree with the repository about the current `HEAD` and about whether uncommitted changes exist:

```
CONTEXT HANDOFF does not name current
Implementer HEAD
CONTEXT HANDOFF disagrees with current
uncommitted changes
```

A successor Implementer will trust this record instead of re-deriving history, so the watcher verifies the record's claims against Git while the evidence is still fresh. This is the general shape of the envelope layer: text grammar first, then identity binding, then agreement with the world.

VIII. THE STORE: ATOMIC CLAIMS AND ADVISORY LOCKS

`mailbox_store.py` owns the filesystem rules that keep two watchers from sharing one mailbox and one message from being handled twice. Its docstring names the two instruments: "the atomic claim that moves a message into the work-in-progress folder without overwriting" and "the advisory file locks ... marking a live watch."

A. The claim: link, fsync, unlink

Moving a file with a plain rename could silently replace an existing file of the same name. The store instead links first:

```
destination = os.path.join(
    directory, os.path.basename(path))
try:
    os.link(path, destination)
except FileExistsError:
    print("  !! refusing to overwrite "
          "existing message state: "
          + destination)
    return None
except FileNotFoundError:
    return None
fsync_directory(directory=directory)
os.unlink(path)
return destination
```

`os.link` creates a second directory entry for the same file and fails—atomically, at the operating-system level—if the destination name exists. Success means this process won the claim; only then is the original entry removed. `FileNotFoundError` on the link means another watcher claimed the message first, and the loser simply moves on: "was already claimed; skipping duplicate dispatch." The `fsync` of the directory forces the new entry to disk before the old entry disappears, so a power loss cannot leave the message with no name at all. A message claimed into `inflight/` and then interrupted stays there and is never re-dispatched automatically; a human reads the record and decides.

B. Advisory locks with honest owners

The mailbox's lock files carry human-readable owner text—the `.dispatch.lock` of a live watch reads `watch pid N`—and the store checks liveness with a deliberately

gentle probe: it tries a *shared, nonblocking* lock. If the probe succeeds, no watcher holds the exclusive lock and the mailbox is unwatched; if it blocks, a live watcher exists. Because the probe is shared, two diagnostic commands probing at once cannot deadlock or mistake each other for the watcher. A second watch attempt on the same mailbox is refused with "another dispatch loop is already running (...); refusing to overlap it."

Two further mode locks, `.fix-only.lock` and `.skip-redteam.lock`, make a watch's policy visible to other processes: a sender can see that a fix-only watch owns the mailbox, and a two-role watch's refusal to accept new Red Team work survives in the lock owner text (`two-role watch pid N`). Both are acquired under the sequence lock, so a concurrent `-send` either publishes wholly before the mode activates or observes the mode marker and refuses—there is no interleaving in which half a policy applies.

C. Topology proofs

Before dispatching into a role worktree, the store validates the complete path topology: the mailbox path contains no symlink redirect inside the checkout, the role worktree is the exact saved path with the exact saved branch, and the shared notes folder is the one the Architect state file names. The result is a *proof* record carrying the resolved paths and their identities, which later phases re-check (`recheck_agent_dispatch_directories`) rather than re-derive—so a path that changes mid-dispatch is caught against the proof instead of silently revalidated against the new world. The same fail-closed style guards the backlog: the store's `_validate_sealed_backlog` refuses a backlog whose bytes differ from the Architect's sealed SHA-256 (Section VI A), and its file reads re-stat the descriptor and the path before and after reading so a swap during the read is detected rather than half-read.

IX. THE WATCH LOOP: RENDEZVOUS, SAFE STOPS, AND CYCLE BARRIERS

A watch runs one lane thread for each enabled role, and a person may press Ctrl-C at any moment. `mailbox_watch.py` coordinates the two with a small concurrency controller, `SafeKillRendezvous`, whose job is to make one printed sentence trustworthy:

```
every enabled role is idle; safe to
Ctrl-C for 19s more; 3 messages waiting.
```

A. Permits and the honest countdown

Every prospective role turn first asks the rendezvous for a *permit*. The permit is a tiny object with three lifecycle flags—launched, reaped, released—and misuse is a hard error ("rendezvous permit launched twice," "rendezvous permit released twice"). The controller counts active attempts and running turns; the safe countdown is printed only when both are zero, and requesting the countdown in any other state is itself an error rather than a wrong message. The most safety-critical rule concerns a child process the watcher launched but could not confirm reaped: the controller then marks itself draining and refuses all later work, on the recorded principle that the watcher must "never advertise safety, or release later work, after losing truthful custody of a child process."

The permit system also paces dispatch. Capacity is reserved across all lanes before a turn starts, so a fast lane cannot start turn $K{+}1$ while turn K is still live; a refused or failed launch frees the reservation, and a reaped child converts it into one completed turn.

B. The watcher watches itself

On every pass the controller compares the modification stamps of `mailbox_daemon.py` *and every part file* against the values recorded at startup:

```
for path, stamp in watched:
    try:
        if os.path.getmtime(path) != stamp:
            changed = True
    except OSError:
        changed = True
if changed:
    self._source_changed = True
    self._draining = True
```

A changed source file drains the watch: running turns finish, nothing new starts, and the watcher exits so a restart loads the new code. This is why editing any part file makes a running watcher stop—a stale watcher enforcing yesterday's rules is worse than no watcher.

C. Cycle barriers

The `-cycle` accounting lives here too. A finite watch may exit only after every admitted ticket completes, and a `-cycle 0` watch may exit only when nothing remains: the exit check holds the sequence lock while it verifies that the backlog has no `- OPEN` line, no ticket cycle is active, no permanent-note transition is pending, and no enabled message was waiting before or after the check. Holding the lock matters: it briefly prevents a concurrent `-send` from publishing a message between "nothing is waiting" and "therefore exit," closing the classic check-then-act race. If any part of that verification cannot be performed, the watcher keeps running instead of guessing that the work is finished.

The same module supplies the argparse validators for the daemon's numeric flags, each refusing with a com-

plete sentence (“cycle count must be a nonnegative integer”; “max characters must use only decimal digits 0 through 9”), and the fail-closed backlog line counter used by the exit check, which applies the same regular-file, size-bounded, identity-stable read discipline as every other backlog reader in the toolkit.

X. CYCLES: THE DURABLE TICKET RECORD

A ticket cycle is one ticket's trip from its first Implementer handoff to its recorded landing, named by joining the ticket's backlog anchor to its full starting commit: `TICKET-ANCHOR@COMMIT`. `mailbox_cycles.py` owns the durable record of every cycle—the JSON state file, the locks that serialize its reads and writes, admission against a finite `-cycle` budget, and the numbered publication of new mailbox messages.

A. The state file

`ai/notes/mailbox/.ticket-cycle-state.json` (schema 6) is the watcher's memory. A fresh state is:

```
{
  "schema": 6,
  "generation": 0,
  "pending_cycle_returns": 0,
  "finite_watch": None,
  "architect_admissions": {},
  "active": {},
  "completed": {},
  "control_plane_history": {},
}
```

Each entry of `active` records one live cycle: its `phase` (exactly one of `implementation`, `committed-awaiting-closure`, or `awaiting-redteam`), its `mode` (`normal` or `two-role`), its landing `commit` (`None` until one exists), and its route, plus the ticket's declared file scope, its class, and—for an Ollama ticket—the pinned Implementer runtime. `completed` maps finished cycles to their landing commits, and a cycle may never appear in both maps at once. The validator that enforces all of this runs on *every* read and write: a state file that fails any rule is a refusal, not a warning, because every later decision would inherit the corruption.

Writes are whole-or-nothing. The writer validates, creates a temporary file with owner-only permissions, writes and `fsync`s the payload, atomically replaces the state file with `os.replace`, and `fsync`s the directory. A crash at any instant leaves either the old complete state or the new complete state—the same discipline the emulator library applies to artifact files, applied to workflow memory.

B. Phase transitions are guarded

A completion is accepted only from the exact expected phase and commit:

```
current = state["active"].get(cycle_id)
if (current is None
        or current["phase"]
            != "awaiting-redteam"
        or current["commit"]
            != accepted_commit):
    raise daemon.TicketCycleStateError(
        "Red Team return does not match "
        "an awaiting ticket cycle")
del state["active"][cycle_id]
state["completed"][cycle_id] = \
    accepted_commit
state["generation"] += 1
```

A Red Team return for the wrong cycle, the wrong landing, or a cycle in the wrong phase cannot complete anything. Registration is guarded the same way from the other side: “the Architect route cannot invent a cycle before an Implementer handoff,” and an Implementer message must cite the public Architect admission that granted its ticket a slot of the finite `-cycle` budget—so a watcher restart cannot admit the same ticket twice, and a forged message cannot conjure a cycle from nothing.

C. Publishing a message without ever overwriting one

Every new mailbox message is published under the sequence lock with the same link-based idiom the store uses for claims, and the comments in the code carry the reasoning:

```
try:
    # Same-directory hard-link
    # publication never replaces a
    # manually created destination or
    # exposes partial bytes.
    daemon.os.link(temporary, path)
except FileExistsError:
    continue
# The state may suppress replay only
# after the directory entry itself
# survives a crash, not merely the
# payload inode.
daemon.fsync_directory(
    directory=daemon.MAILBOX)
return path
```

The payload is fully written and synced under a temporary name first; the hard link claims the numbered name atomically; a collision simply tries the next number; and only after the directory entry is durable may any state record assume the message exists. After twenty failed

attempts the publisher gives up and asks a human question: "could not claim a sequence number after 20 tries; is something flooding the mailbox?"

XI. DISPATCH: LAUNCHING AND SUPERVISING ONE ROLE TURN

`mailbox_dispatch.py` owns the lifecycle of one claimed message: launch the role's command-line program in the role's own worktree, watch it, kill it if it exceeds the timeout, and verifiably move the finished request to `done/` or `failed/`. Its boundary is stated in its docstring: "It does not interpret the returned text; the envelope part matches returns to their tickets."

A. The command a role actually runs

The daemon builds each role's argument list once per run (`build_agent_commands` in `mailbox_daemon.py`). The Architect's Claude session, for example, is launched headlessly as

```
[CLAUDE_EXECUTABLE, "-p",
 "--no-session-persistence",
 "--model", architect_model,
 "--effort", fable_effort,
 "--permission-mode", "acceptEdits"]
```

while the Red Team's Codex session pins its model, effort, service tier, sandbox, working folder, and shared-notes access on the command line, and an Ollama Implementer is launched through `ollama launch claude`, which supplies the same coding-tool shell with an Ollama-served model doing the reasoning. Three facts deserve attention. `-no-session-persistence` (and `-ephemeral` for Codex) means every turn starts an empty conversation: an old ticket cannot leak context into the next one. The permission mode and service tier are *not* command-line options of the daemon; they are fixed in code and changing them is a reviewed code change. And the three executable paths at the top of `mailbox_daemon.py` are the single machine-specific block in the whole toolkit—on a new computer they are the only edits.

The prompt itself is assembled by concatenation: a banner naming the run's policies, the role's preamble, any admission prompt, the common preamble, and then the raw message body as the exact suffix. The body was validated as an envelope before launch; a body containing a NUL byte is parked with the printed reason that it "cannot be a command argument."

B. Supervision: process groups and honest timeouts

The child is started with `start_new_session=True`, which makes it the leader of a new process group. This is the mechanism behind a promise the user documentation makes: a timeout kill stops the AI program *together with every helper program it started*, because the kill targets the group:

```
try:
    daemon.os.killpg(
        proc.pid, daemon.signal.SIGKILL)
except (AttributeError, TypeError,
        ProcessLookupError,
        PermissionError, OSError):
    proc.kill()
proc.wait()
```

The monitor loop polls every five seconds, prints a periodic progress line naming the relay log, and at the deadline polls *once more* before killing—a child that finished naturally during the last sleep is reaped, not murdered. A killed turn writes a timeout-history record under `.dispatch-history/` and its request is parked in `failed/`.

C. The verified move

Moving a finished request to `done/` is itself treated as an operation that can fail halfway. Before the move, the dispatcher holds a same-inode hard-link guard; after the move it verifies that the destination has the source's inode *and* the source name is gone. If either check fails, the guard link restores the original state and the request stays where inspection will find it. A role process that exited successfully is still not marked finished until this verification passes—the difference between "the program said it finished" and "the record of finishing survived."

D. The authority tripwire

Immediately before an Implementer turn the daemon records local `main`, `origin/main`, and the user checkout's branch, commit, and status; immediately after, it compares. Any movement raises `ImplementerAuthorityViolationError`, prints

```
IMPLEMENTER AUTHORITY VIOLATION:
- local main changed during the
  Implementer turn.
Candidate or partial work preserved
in WORKTREE; nothing landed.
```

parks the request, and exits the watch. The check detects that repository state moved during a turn that had no business moving it; the documentation is explicit that it cannot attribute the movement or defend against a hostile process running as the same operating-system user.

E. Providers, health, and runtime pinning

Three helpers surround the launch. `provider_health.py` answers "can the selected services respond at all" by sending each one a private nonce marker and requiring the reply to equal it exactly—a model that answers anything but the marker line fails the ping. The Ollama check is stricter: it launches the real `ollama launch claude` integration inside a disposable `git init` repository (so the probe cannot inspect or edit the checkout), reads the model's reported context length, and refuses a context that is unverifiable, below 32,768 tokens, or smaller than the requested compaction point.

`mailbox_providers.py` then pins the verified choice. When a ticket begins, its cycle record stores the provider, model, verified context, and compaction point; a restart with different values stops and names the saved ones. The rule in the docstring: "a restart must not silently change that choice in the middle of a ticket." An unfinished Ollama ticket cannot quietly become a Claude ticket.

`review_dispatch.py` implements the cheaper-review policy. It classifies a turn as one of six routine review kinds (a Red Team closure, an Architect reopening decision, a candidate audit, and so on) and swaps exactly the reasoning-effort token in the already-built command—refusing to guess if the command does not contain exactly one effort slot. Authority is untouched; only the price of a bounded recheck changes.

Finally, `implementer_checkpoint_hook.py` is not run by the daemon at all: it is registered as a Claude Code hook inside the Implementer's session. After a tool batch or at session end, once a monotonic deadline (90 minutes, read from the role contract) has passed, it instructs the Implementer to commit clean progress and write a checkpoint handoff; at context compaction it instead demands the full context-handoff record of Section VII. A tiny state file under an exclusive `flock` makes the instruction one-shot even if the harness fires several hooks at once, and a subagent event (recognized by its `agent_id`) is ignored so helpers do not trigger the pause meant for the main session.

XII. THE HANDOFF CONTRACT: A DIRECTIVE A MACHINE CAN CHECK

The largest file in the toolkit, `handoff_contract.py` (2,469 lines), validates the note an Architect or Red Team writes before handing work to the Implementer. Its docstring bounds its ambition precisely: "It checks structure and obvious placeholders; it cannot decide whether the scientific design itself is correct." Its size is the honest cost of the protocol's central bet: if the Implementer may be a smaller model, the plan must be *decision-complete*, and decision-completeness must be mechanically checkable because no human proofreads role-to-role mail.

A. The packet and its fifteen sections

The whole directive lives under exactly one level-two heading— `## Implementation directive` for the Architect, `## Repair directive` for the Red Team—and must contain exactly these level-three sections, in exactly this order (the Architect's list; the Red Team's thirteen-section list is analogous):

> Outcome; Starting point; Execution checkout; Character-change budget; Role plan; Files and symbols; Ordered implementation steps; Interfaces and exact behavior; Failure behavior and edge cases; Tests to write; Validation commands; Acceptance checklist; Do not change; Stop and ask if; Parallel work plan.

A missing, extra, renamed, or reordered heading is refused with a message that lists both the required and the found order. Order matters because each section is parsed by its own specialist validator, and because a directive that reads in execution order is a directive the Implementer can follow top to bottom.

The structured sections have row-level grammars. *Execution checkout* is exactly three rows—worktree (one literal absolute path with no shell metacharacters), branch (never `main`), and base (exactly 40 lowercase hexadecimal characters). *Character-change budget* is exactly three rows—the limit (which must equal the run's `-max`), the planned maximum (which must fit under the limit), and a substantive readability plan. *Role plan* is exactly the four rows the READMEs show, with cross-checks: a plan that includes the Red Team must select a real severity and scope; a plan without one must say `not-used` for both; a `widespread` review forces severity `low`; and the `protected-control-plane` class is refused outright as an Implementer ticket class. Every edit bullet in *Files and symbols* and every test bullet must begin with a locator of the form `‘repo/path::symbol‘` followed by a substantive description—generic placeholders such as `path/to/file` or a bare `symbol` are rejected, and an Architect locator under `ai/tools/` is refused with the full policy sentence: "ai/tools/ is external-maintainer-only; keep the backlog ticket Open and do not send it to the Implementer."

Validation commands must contain a closed shell fence whose first real command parses under `sh -n` and resolves to a builtin, a program on `PATH`, or an executable file—a directive cannot demand a command that cannot run. When a character budget is active, the section must also contain exactly one literal `ticket_change_guard.py` invocation whose `-max`, `-repo`, and `-base` match the budget and checkout rows, plus a matching "within limit" checkbox in the acceptance list: the contract stitches the guard of Section VI C into every sized ticket.

B. The adversarial-Markdown posture

The validator treats the note as hostile input, in the same spirit as the envelope parsers. It masks frontmatter, strips HTML comments outside code fences, and refuses Setext headings, fences nested in lists, display math, raw HTML, invisible control characters, and inline links in binding fields ("binding directive fields may not use inline Markdown links or images; write visible prose instead"). Each of these is a way to make a human and a parser read different documents; refusing them keeps one reading.

The most distinctive check is the unresolved-choice detector. A directive that delegates a design decision—"use your best judgment," "choose either A or B"—is refused with the offending phrase quoted: "section … delegates an unresolved design choice with …." A genuine machine rule ("use A if the file exists, otherwise B") passes. The line between a condition and a choice is the line between a plan and a wish, drawn in a regular expression.

C. Helpers are planned, and evidence must match the plan

The *Parallel work plan* section has three legal shapes: a `#### Subagents required` plan with a launch row and one block per helper (each with the ordered fields Mode, Ownership, Task, Return, Acceptance, Stop, closed by an Integrator block); a `#### Subagents not required` declaration with a concrete reason; or a capability exception recording exactly what launch was attempted and how it failed. On the way back, the Implementer's handoff must contain one `Subagent return` block per planned helper, and the match is strict:

```
if returned != planned:
    raise DirectiveError(
        "IMPLEMENTER_HANDOFF Subagent "
        "returns must match the planned "
        "names and order; planned "
        + repr(planned) + ", returned "
        + repr(returned))
```

Missing, renamed, reordered, or extra helper results are all visible in that one comparison. An Implementer that skipped the planned fan-out cannot write a passing handoff by prose alone.

D. Two loaders, one authority

The contract is consumed two ways. The router imports it as ordinary Python. The daemon, however, loads the *authoritative* copy by absolute path—pinned through the `MAILBOX_HANDOFF_CONTRACT` environment variable—and asserts it exposes the required API names, for the same reason the character guard checks its own path: role worktrees contain copies of every tool, and a stale copy validating yesterday's grammar would be an invisible downgrade. Reading a directive file is itself defensive: the file is opened with `O_NOFOLLOW`, its identity is recorded before and after reading, and a mismatch refuses with "directive note changed while it was being read."

XIII. THE MANUAL RELAY: `HANDOFF_ROUTER.PY`

The mailbox daemon automates a workflow that can also be driven by hand: an Architect in one chat window, an Implementer in another, and a human clipboard between them. `handoff_router.py` exists so that even the manual mode has machine-checked custody. Its docstring fixes the human's job title: "A human courier may paste a generated handoff block unchanged into the Implementer or Red Team conversation." Courier—not editor, not author.

The relay run (`-note ai/notes/ticket.md`) validates the directive with the handoff contract, binds itself to the directive's declared worktree, branch, and base (refusing to run anywhere else), and then drives the clipboard: it copies the generated `ARCHITECT_HANDOFF (relay)` block, prints a numbered progress line, and polls the clipboard for the reply:

```
while True:
    current = read_clipboard()
    if (current != last_copied
            and has_handoff_header(
                current, header)):
        return current
    time.sleep(0.5)
```

The two-condition test is the whole trick: a block is accepted only when it carries the expected heading *and* differs from the text the router itself last copied, so the router cannot mistake its own prompt for an answer. A machine-wide file lock serializes clipboard use—two concurrent relays would otherwise steal each other's blocks—and every run is journaled as a crash-recoverable route record, so an interrupted relay resumes rather than repeating paid work ("an unfinished route names a different note, version, or check command list" is the refusal when a resume does not match).

Authority stays where it belongs. The router's own generated audit prompt tells the Architect, verbatim: "Before GO, close and seal this ticket in the backlog. Do not merge, commit, update main, or push; save the decision as a to-daemon message and let mailbox_daemon.py create and record landing L." The router's `-skip-redteam`, `-mode`, and `-severity` options can only *confirm* what the validated note already says; a mismatch is refused before the clipboard changes. And the widespread-review refusal is fail-closed against the authoritative backlog: one malformed open ticket line is enough to block a widespread search, because an unparseable ledger cannot prove the required "zero open Critical, High, or Medium" precondition.

`-status` is the read-only face: it summarizes saved work, finished and waiting reviews, and the suggested next action, and is safe to run at any time.

XIV. WORKTREES: PROVISIONING, ADOPTION, AND THE BACKLOG CARRY-FORWARD

`mailbox_worktrees.py` (2,715 lines, the largest part) creates or safely adopts the role folders of Section III, records each folder's identity in the JSON state files, validates every identity before a dispatch, and owns `-clean-all`. Three of its behaviors teach the system's philosophy better than any summary.

A. Adopt, never reset

Provisioning distinguishes "create fresh," "adopt an interrupted bootstrap," and "refuse." A registered worktree at the expected path on the expected branch whose state file was never written is recognized as an interrupted bootstrap and adopted—the state file is completed and the code prints "recovered exact interrupted Implementer-worktree bootstrap." A registered worktree on the *wrong* branch is refused ("default Implementer path is registered on another branch"), and a branch that exists without its registered worktree is refused with the design principle in the message itself: "refusing to reset or reuse it." Nowhere in provisioning is there a `git reset` of something that might be a person's work; ambiguity is always an error message naming what was found, never a guess.

The same module rechecks the role contract on every pass ("role contract changed after daemon startup; restart before admitting more work") and refuses an automatic topology migration: a state file from the era before the separate Implementer worktree stops the daemon with instructions, because migrating live state under a possibly-running old daemon is exactly how two programs corrupt one mailbox.

B. The backlog rides the fast-forward

When the Architect's coordination worktree advances to a new landing, one tracked file needs special care: the backlog, which the Architect may have edited and sealed *after* the commit being advanced to was created. The carry-forward uses Git's three-way file merge (`git merge-file`) with the old base, the incoming `main` version, and the Architect's sealed version; a genuine conflict—both sides editing the same text—refuses with both versions preserved ("main and the sealed Architect backlog edit the same text; both versions remain preserved for manual reconciliation"). On success the merged backlog is restored over the fast-forwarded worktree and *resealed*: the guard state file is rewritten with the new SHA-256 and the previous one, keeping the seal chain of Section VI A unbroken across the move. A sync-recovery file holds the merged bytes during the transition, so a crash mid-carry leaves evidence instead of a mystery.

C. `-clean-all` is bounded destruction

The one deliberately destructive command refuses to run from anywhere but the user's main folder, refuses while any watcher or sender is live, and then removes *only* managed children of `.claude/worktrees/` and worktrees on AI branches (`claude/*`, `codex/*`, legacy `worktree-agent-*`). A managed child on a non-AI branch is explicitly preserved; branch deletion touches only names passing the AI-branch test; the exit line promises what happened: "clean-all finished; main and non-AI branches were not changed." Destruction, like everything else here, is scoped by an allowlist rather than a hope.

XV. CANDIDATE TO LANDING

This is the chapter the whole system exists to make safe: how the Implementer's proposal becomes the accepted tip of `main`. `mailbox_landing.py` owns it. The focused companion guide `documentation/candidate_to_landing.pdf` teaches the same mechanism at Git-object level with worked commands; this section explains the code that implements it. (One bookkeeping note for grep users: these functions are addressed as `mailbox_daemon.`⟨`name`⟩ but written in `mailbox_landing.py`, per Section IV.)

A. Recording candidate C durably

When an Implementer turn returns clean, the daemon preserves the candidate commit under a private Git reference, `refs/mailbox/cycles/`⟨`sha256(cycle)`⟩`/candidate`, so ordinary branch movement can never garbage-collect it. The update is a Git-level compare-and-swap: `git update-ref ref new expected` succeeds only if the reference still holds the expected old value, so an interrupted write cannot clobber a different candidate. A JSON state file mirrors the map from cycles to references and commits, written with the same atomic replace idiom as the cycle state. Sibling references under the same hash-named folder journal the prepared landing and any reopen decision—these references *are* the crash journal that recovery reads.

B. Building landing L

After a `GO`, the watcher builds L without ever checking files out, using Git's plumbing directly. First the squash tree:

```
result = _run_git(
  arguments=["merge-tree", "--write-tree",
             parent_commit,
             candidate_commit], ...)
if result.returncode != 0:
    raise TicketCycleStateError(
        "the audited candidate does not "
        "squash cleanly onto the "
        "landing parent")
```

`merge-tree -write-tree` computes the tree that would result from merging the candidate onto the current `main` tip, entirely inside the object database; a conflict is a refusal, never an auto-resolution. Second, the sealed backlog blob is overlaid into that tree through a scratch index, so the landing carries the Architect's closed-and-resealed backlog in the same commit as the fix—one commit per accepted ticket, bookkeeping included. (The companion guide's tree T is the pure squash; the code's tree is T plus the sealed backlog. The distinction matters when comparing the two documents.) Third, the commit itself:

```
["commit-tree", tree,
 "-p", current_main, "-F", "-"]
```

with exactly one parent—the role contract's `parent_count: 1` made executable—and a message that is the candidate's own Architect-approved message plus two appended machine trailers:

```
Mailbox-Cycle: TICKET-ANCHOR@COMMIT
Mailbox-Candidate: FULL-CANDIDATE-COMMIT
```

The trailers make every landing self-describing: after any crash, the commit alone names its ticket and its candidate. They are reserved labels—a candidate message that already contains them is refused rather than allowed to forge its own provenance, and varying the letter case or colon spacing does not slip past the check. Before the prepared landing is ever used, `_verify_prepared_landing` rebuilds the expected tree from the candidate and compares: "prepared landing tree is not the exact candidate squash" and "prepared landing message does not bind its cycle and candidate" are the refusals. Nothing journaled is trusted without being re-derived.

C. Installing L in the user's checkout

The installation is a fast-forward or nothing:

```
if _user_checkout_status():
    raise RetryableArchitectLandingError(
        "the user's main checkout has "
        "staged, unstaged, or untracked "
        "work; exact landing was "
        "preserved without touching it")
result = _run_git(
    repository_root=REPO_ROOT,
    arguments=["merge", "--ff-only",
               landing], check=False)
```

followed by a re-check that `HEAD` now equals L and the checkout is still clean. `-ff-only` can only advance the branch along existing history; it cannot rewrite, merge, or discard anything. Note the exception type: a dirty checkout is *retryable*—clean it and restart—while a verification failure after the merge is a hard stop. The error taxonomy (retryable versus fatal landing errors, plus the authority-violation and token-exhaustion classes) is how the watch loop knows whether to park, retry, or die.

Before this step runs, `_prepared_landing_main_problem` screens the world for the three ways it can have moved since L was prepared. If `main` already contains L with newer commits on top, the situation is recovery, not landing. If `main` preserved L's parent M0 but advanced to a newer M1, the diagnosis line `STALE_INTEGRATION_REVALIDATION: C=... L=... M0=... M1=...` is emitted, and the Architect performs the *narrow* check the READMEs describe: does the M0-to-M1 change affect C? If `main` no longer descends from M0 at all, history needs the user, and the code says so. In all three cases C, the `GO`, and L are preserved untouched.

D. Push, or debt

After a verified local landing, exactly one non-force push attempt is made, and the result is verified against the remote itself: a `git ls-remote` must report exactly the landing commit at `refs/heads/main`. Any failure or uncertainty writes a push-debt record, `ai/notes/relay/pending-main-push-⟨L⟩.txt`, whose body names the exact commit and the exact command still owed:

```
Local main contains verified landing L.
Push is still required:
  git push origin L:refs/heads/main
Last push result: DETAIL
```

The debt is never repaid by rebuilding or re-landing; a later watch retries the recorded push. With `-github no` the push is skipped as a user choice, no debt is recorded (nothing is owed), and older debt records stay on disk for a later `-github yes` watch.

E. Audit snapshots

The Architect audits C, and the Red Team reviews L, through temporary *detached* worktrees named `mailbox-audit-⟨hash⟩-⟨agent⟩` under the managed root. Detached means no branch: the snapshot shows exactly one commit, later Implementer work cannot change what the reviewer is reading, and the snapshot is removed when the review ends. This is the mechanism behind the README's promise that reviews never inspect a proposed change through newer unfinished files.

XVI. RECOVERY: RESUMING INSTEAD OF LOSING WORK

Any run can stop between two steps: a crash, a timeout kill, or Ctrl-C. `mailbox_recovery.py` owns the machinery that makes the next start *resume*—and, just as deliberately, the two explicit commands that *discard*. The distinction between the two is the chapter's central lesson.

A. Resumable means durable evidence exists

Recovery replays only what disk can prove. A recorded Architect `GO` whose archive step never ran is re-consumed: the recovery path re-parses the envelope and calls the same landing orchestration, which "re-proves the landing's binding to this cycle and candidate" rather than trusting the journal. A validated Implementer return that was never delivered is finished from its delivery receipt—a hard-link record created *before* the request was archived, whose name binds the request and return files to their SHA-256 digests. An interrupted mailbox move is diagnosed by inode identity: the claim sequence (link into `inflight/`, fsync, unlink the root copy) leaves a characteristic debris pattern at each possible crash point, and the recovery code walks the possibilities, finishing moves whose evidence is consistent and refusing with "interrupted mailbox claim has conflicting states" when it is not. In every resumable case the principle is the same: the work was already paid for, the evidence survives, so the transition is completed—without rerunning any AI turn.

B. Discarding is explicit and guarded

`-restart-implementer` and `-restart-redteam` are the only paths that throw partial work away, and each refuses when the work already produced something the Architect owns:

> "the Implementer already produced candidate C; return it to the Architect instead of restarting"
> "the Implementer already returned work for this cycle; send it to the Architect instead of restarting"

Only when exactly one active handoff exists and no candidate or return does, the Implementer worktree is reset to the ticket's base and cleaned, the reset is verified ("Implementer work could not be discarded cleanly" otherwise), and the exact saved handoff is requeued. The Architect's plan—the expensive part—is never regenerated.

C. Deferring Ctrl-C across a durable transition

The backlog pass wraps each state transition in a context manager that defers `SIGINT`:

```
def __exit__(self, exc_type,
             exc_value, traceback):
    if self._installed:
        signal.signal(signal.SIGINT,
                      self._previous)
        if self._pending \
                and exc_type is None:
            raise KeyboardInterrupt
    return False
```

A Ctrl-C pressed while a landing sequence is midway is recorded, the sequence finishes whole, and the interrupt is honored at the boundary. Combined with the watch-level two-press protocol (Section IX) this is why the user documentation can say that Ctrl-C never corrupts saved work: interrupts are not blocked, they are *scheduled*.

D. The master recovery order

Before any live dispatch, one function runs the whole recovery family in a fixed order: interrupted moves first, then parked Architect outcomes, parked `GO`s, failed Implementer returns, undelivered deliveries, failed preflights, prelaunch parks, failed public admissions, blocked Red Team messages, and finally a reconciliation of the ticket-cycle state against the world. The order is part of the correctness argument—each later step may depend on the files an earlier step repaired—and it runs on *every* start, so "restart the watcher" is the universal first-line remedy the READMEs promise.

XVII. THE PROTECTED BOUNDARY AND THE TWO-KEY RITUAL

Two kinds of change are too dangerous for the ordinary ticket path: changes to the workflow's own program files, and changes to the protected policy files the program obeys. The system treats them differently—one is banned outright, the other gets a heavier ritual—and reading the difference teaches its threat model.

A. ai/tools/ never lands from inside

No mailbox ticket may change a file under `ai/tools/`: a workflow must never propose and land its own replacement. The rule is enforced at three independent points, each of which a reader has now met: the handoff contract refuses an Architect directive whose locators enter `ai/tools/` (Section XII); candidate admission reports `PROTECTED_PATH_VIOLATION` for any such changed path (Section VI D); and landing preparation independently rechecks and refuses with "external-maintainer-only ai/tools change cannot land" (Section XV). Three points, three different modules, so a single defective check cannot open the door. A Red Team audit may still *report* a tools bug; the ticket is recorded and waits for an external maintainer working outside the mailbox.

B. Protected notes: the Architect-only route

Changes under `ai/notes/`—the eleven permanent notes, the role contract, the role files—use the guarded exception instead. The Architect queues the update from inside an already-running Architect process (`-architect-notes-admin`); the daemon marks the job with `MAILBOX-ADMIN: permanent-notes` and supplies the base commit B through the environment; the Architect commits P, a clean commit whose *only* parent is B and whose only changed paths are protected policy files; and the return envelope is the four-line record of Section VII naming B, P, and `GO`. The watcher verifies the pair—parenthood, changed-path census, idle role folders—then fast-forwards `main` to P and updates every safe role folder, so no role starts from older rules. A journal under `ai/notes/relay/` tracks the phases (`started`, `validated-noop`, `validated-commit`), bound by SHA-256 to the exact request bytes, so this path too resumes after a crash. No cycle is consumed: policy maintenance is not a ticket.

C. Protected control-plane tickets need two keys

For the rare protected ticket class that remains (Architect-owned policy work flowing through the normal candidate machinery), `mailbox_control_plane.py` keeps a two-key record: landing requires the Architect's `GO` *and* the Red Team's separate `ACCEPT-CONTROL-PLANE` decision, both recorded against the same exact candidate:

```
def control_plane_keys_ready(
        control, candidate_commit):
    return (isinstance(control, dict)
        and FULL_COMMIT_RE.fullmatch(
            candidate_commit) is not None
        and control.get(
            "architect_candidate")
            == candidate_commit
        and control.get(
            "redteam_candidate")
            == candidate_commit
        and control.get("redteam_result")
            == "ACCEPT-CONTROL-PLANE")
```

An Architect decision that names a different candidate than an earlier recorded key is refused ("protected Architect decision changed candidate C")—the keys cannot drift onto different commits. The record also carries shadow-check and health fields: before a protected landing, the daemon runs a *shadow validation* that exercises the candidate's daemon in a throwaway repository and proves the live state was untouched, and a failed health check makes the running watcher recovery-only until a human looks.

D. Replacing the watcher itself: D0 rehearses D1

The deepest protection concerns the daemon's own succession. Before an external maintainer lands a replacement controller D1, the current controller D0 rehearses the takeover using `control_plane_handoff.py`: D0 copies its live records—ticket state, candidate state, worktree records, recovery files, private references—into a disposable checkout, then runs a D0-authored probe *with D1's code* in a fresh process. The probe re-reads everything and asserts that D1's interpretation equals D0's saved meaning, field by field and digest by digest, printing `DO_TO_D1_STATE_HANDOFF_PASSED` only at the end. If D1 changes any state schema, it must ship an explicit migration declaration naming the old schemas, new schemas, migration function, and preserved facts; the probe refuses "state schema changed without an explicit migration declaration" otherwise, and the migration runs only on the disposable copies. The design rule appears in the docstring: "Candidate tests never decide whether the takeover succeeds"—D0 defines the expected meaning, D1 merely re-derives it. A clean-start test would prove D1 can begin; this proves D1 can *inherit*.

XVIII. TICKETS, THE BACKLOG, AND REOPENINGS

The tracked ledger `ai/notes/backlog.md` is the system's queue, and several modules read it under one exact grammar. This section collects the lifecycle rules a reader needs to interpret those modules.

A. The ledger grammar

An open ticket is one index line plus one anchored body. The index line must match, character for character,

```
- OPEN **SEVERITY** **KIND** -
  [title](#anchor)
```

(on one physical line) with `SEVERITY` one of `CRITICAL`, `HIGH`, `MEDIUM`, `LOW` and `KIND` one of `BUG FIX`, `NEW FUNCTIONALITY`. The body begins at the HTML anchor `<a id="anchor"></a>` followed by a `##` title, ends at the next anchor or heading, and carries bold status rows—`**OPEN.**` or `**CLOSED.**`, `**Severity: ...**`, `**Red Team reopen count: N.**`, and `**Red Team reopening: allowed.**` or `**Red Team reopening: barred by Architect NO-GO.**`—each matched by its own anchored regular expression. Whether a ticket is open is decided *structurally*: its anchor sits above or below the single `# Closed tickets` heading, and the readers cross-check that structural position against the status rows and the index, refusing on disagreement ("Open ticket index and detail severity disagree"; "Closed ticket still appears in the Open index"). A line that fails the grammar is not ignored: it is counted as *unclassified*, and unclassified lines block new discovery until the Architect repairs them—a formatting mistake must not hide work from the limits.

One consequence deserves emphasis because it once cost a real debugging session: the body-boundary scan recognizes only `<a id>` anchors and `##` headings. A closed ticket followed by an anchor-less section can therefore appear to extend that ticket's body, and the reopening-status helper will refuse the decision. The grammar, not the visual layout, is the truth.

B. Discovery is rationed

A discovery—asking the Red Team to look for a *new* bug—must clear a series of gates implemented in `sol_ticket_refusal`, each with a complete refusal sentence. Fix-only mode forbids discovery outright ("fix-only watch is closing-only ..."). Ten or more open Critical, High, and Medium tickets defer it, with instructions to record the candidate finding locally "without a countable `- OPEN` marker." A widespread search is automatically Low and additionally waits until the non-Low count is zero. The severity and scope travel *in the saved request* (`MAILBOX-SEVERITY`, `MAILBOX-SCOPE` as exact second and third lines), so a later watch with a different default cannot reinterpret a stored discovery; a stored request missing either line is refused rather than defaulted—the never-trust-defaults rule applied to workflow state.

C. Fix-only maintenance

A fix-only watch turns the system into a bug-closing machine: the user queues one general request, the Architect must select the first eligible open `BUG FIX` at or above the watch's severity in ledger order, and after each Implementer handoff the daemon requeues the same general request for the next bug. Eligibility is computed from the ledger by the same fail-closed readers (`eligible_fix_only_bug_anchors`), and when nothing eligible remains the watch exits idle with a sentence saying exactly that.

D. Reopenings: mechanical facts, human judgment

When the Red Team returns `REOPEN`, the Architect must decide, and `reopen_transition.py` brackets that decision with pure bookkeeping. Before: `inspect_backlog` extracts the ticket's exact state (severity, count, open or closed, reopening allowed or barred) and `architect_brief` prints the checked facts and the two legal outcomes, so the Architect spends reasoning on evidence rather than ledger arithmetic. After: `validate_after` accepts only an exact legal transition—

```
if after.count != before.count + 1:
    raise ReopenTransitionError(
        "Architect decision must "
        "increment the reopen count "
        "exactly once")
```

and the outcome must be exactly `OPEN`+allowed (a `GO`) or `CLOSED`+barred (a `NO-GO`), in both cases at the expected severity, where "expected" encodes the escalation brake: from the sixth reopening onward the ticket is forced to `LOW` (`return "LOW" if ticket.count + 1 > 5 else ticket.severity`). The module "never edits the backlog and never decides whether the Red Team is right"—it is the same facts-versus-judgment split as every guard, applied to the one transition where a lost or double-counted decision would silently corrupt the ledger's history. Idempotence appears here too: a decision already recorded (the exact after-state on both sides) is recognized as a harmless replay rather than refused.

XIX. BUNDLES: MOVING UNFINISHED WORK BETWEEN PEOPLE

The backlog and its local supporting notes are deliberately not all in Git; when one person must hand unfinished work to another, `backlog_bundle.py` packages them. Its docstring sets the posture: the archive "is an email attachment, not a Git artifact," and incoming archives "are treated as hostile."

The pack side collects the backlog, local notes, support files, and explicit inclusions into a deterministic USTAR tar inside a single XZ stream, first member a canonical-JSON manifest recording the format, the base Git commit, the repository identity, every file's SHA-256 and size, and the ledger's open items. Protected knowledge—the eleven notes, the role contract—is *not* packed; it is anchored by the recorded base commit, and packing refuses

while any protected file differs from that base ("protected knowledge must come from the recorded Git base"). The recipient gets those files from Git, where tampering is visible, not from an attachment, where it is not.

The unpack side validates before returning a single byte: the XZ stream must be exactly one stream with no trailing data, every tar header must be byte-identical to the one canonical header the packer would have written ("only canonical USTAR headers are accepted"; "links, directories, devices, and tar extensions are forbidden"), the manifest must be the first member and in canonical JSON, the member list must exactly equal the manifest, every payload file must match its recorded size and digest, and the manifest's open-item list must match the packed backlog's actual bytes. Extraction writes into a fresh Git-ignored folder under `ai/backlog-imports/`, owned during the copy by an `.INCOMPLETE` marker and completed by a `.COMPLETE` marker, resumable only when every already-written byte matches expectations. Nothing ever overwrites live notes; import is inspection material, not an automatic merge.

For a reader, the module doubles as a catalog of safe-parsing idioms: canonical encodings compared byte-for-byte, allowlisted structures rather than blocklisted attacks, sizes capped before reads, and markers that make partial work distinguishable from finished work.

XX. A FILE-BY-FILE ROUTE FOR STUDYING THE CODE

The dependency structure suggests a reading order that never requires a forward reference. Each stage's files can be read completely with only the previous stages in mind.

A. Stage 0: orientation before Python

Read `ai/README.md` (the operating guide: roles, cycles, candidate C and landing L in user language), then `ai/tools/README.md` (the command guide: every flag, every recovery appendix), then the five-page `documentation/candidate_to_landing.pdf`. Together they supply every term this manuscript defined in Sections I–II.

B. Stage 1: the trust spine (small, standalone files)

1. `role_contract.py` (378 lines) — the safety floor, the canonical-format check, the duplicate-key hook. Every idiom here recurs.
2. `candidate_admission.py` (41 lines) — the purest module: policy as set arithmetic.
3. `backlog_guard.py` (516) — the seal chain, the paranoid file read, the crash-retry no-op.
4. `permanent_note_guard.py` (413) — the four-state comparison, the exact census, the guard checking itself first.
5. `ticket_change_guard.py` (811) — exact character counting, work budgets, the audit-versus-self-check boundary.
6. `reopen_transition.py` (217) — the ledger grammar as regular expressions, legal-transition validation.

After stage 1 the reader has met, in miniature, every defensive idiom the daemon uses at scale.

C. Stage 2: the daemon's skeleton

1. `mailbox_daemon.py`, top half — the docstring, the constants block (every state-file name and schema number lives here), `_load_local_tool`, `_DaemonNamespace`, `_adopt_daemon_part`, and `MAILBOX_PART_FILES`. Do not read `main` yet.
2. `mailbox_envelopes.py` — the header vocabulary, one envelope parser end to end (the ticket-flow one), one return matcher end to end (the `GO` one), and the inode-snapshot helpers.
3. `mailbox_store.py` — `move_without_overwrite`, `claim_message`, the lock probes, one topology validator.
4. `mailbox_watch.py` — `SafeKillRendezvous` in full; it is the only concurrent code in the system.

D. Stage 3: one running turn

1. `mailbox_dispatch.py` — `dispatch_under_main_checkout_lock` straight through: it is long but linear, and every phase was previewed in Section XI.
2. `provider_health.py`, `mailbox_providers.py`, `review_dispatch.py`, `implementer_checkpoint_hook.py` — the provider surround, each small.

E. Stage 4: the commitment path

1. `mailbox_cycles.py` — the state schema and `register_ticket_cycle_message`; then `publish_message_locked`.
2. `mailbox_landing.py` — candidate refs, `prepare_exact_squash_landing`, the clean-checkout fast-forward installer, and `push_exact_landing_or_record_debt`.

3. `mailbox_worktrees.py` — provisioning and the backlog carry-forward; skim the legacy-transport helpers on a first pass.

4. `mailbox_recovery.py` — `recover_before_dispatch` and `process_backlog`; then return to `mailbox_daemon.py`'s `main` and the watch loop, which now reads as a thin driver over familiar functions.

5. `mailbox_control_plane.py` and `control_plane_handoff.py` — the two-key record and the D0/D1 rehearsal, best read last because they reuse everything.

F. Stage 5: the human-driven tools

`handoff_contract.py` and `handoff_router.py` close the loop: after the daemon, the contract's grammar reads as the mirror image of the envelope layer, and the router as a hand-cranked dispatch. `backlog_bundle.py` stands alone and can be read at any point after stage 1.

G. The tests are executable claims

The witnesses under `ai/tests/` are part of the documentation. Because of the namespace design (Section IV), a test patches `mailbox_daemon.`⟨`name`⟩ and every part sees the replacement; reading a few such tests shows exactly which collaborators a function is specified against. The repro scripts (files named `tools_*_repro.py`) go further: each builds a disposable Git repository, drives one workflow scenario end to end, and asserts the printed verdicts—the candidate-and-landing distinctness check quoted in `ai/README.md` is one of them. When this manuscript and the code ever disagree, run the witness; it is the statement with an exit code.

H. Three shorter routes

The operator (wants to run watches safely): stage 0, then Sections III, IX, and XVI, plus the two guards they will run by hand (`backlog_guard.py`, `permanent_note_guard.py`).

The auditor (wants to trust a landing): stage 1, then Sections VII, X, and XV, then `mailbox_landing.py` itself with `candidate_to_landing.pdf` open beside it.

The contributor (must change a tool as external maintainer): the whole route, then Section XVII again, then the D0/D1 rehearsal in `control_plane_handoff.py`—because their change is exactly the case that machinery exists to survive.

XXI. CLOSING: THE SYSTEM IN ONE PARAGRAPH

Every agreement is a file with one writer. Every file has a grammar, and every grammar is enforced where the file is read, not where it is written. Every transition is atomic, journaled, or refused; every journal is re-proved before it is trusted; and every refusal is a complete sentence naming what was found and what was required. Authority lives in a validated contract, not in whichever process speaks first; models are launched, supervised, and forgotten; and the one irreversible action—advancing `main`—is a fast-forward to a commit that names its own ticket, its own candidate, and its own sealed ledger. A reader who internalizes those sentences can predict most of the 27,900 lines before opening them; the lines themselves supply the exact spellings, and the tests supply the proof.